

# S<sup>2</sup>MoE: Spiking-based Mixture-of-Experts for Scientific Machine Learning

Shock/Smooth Spatial Specialization for Autoregressive PDE Rollout

*A Self-Contained Technical Report*

Methods, Background, Equations, Protocol, and Results

## Contents

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction and problem statement</b>                   | <b>1</b> |
| 1.1      | What is a PDE surrogate? . . . . .                          | 1        |
| 1.2      | Why spiking neurons? . . . . .                              | 2        |
| 1.3      | Why a Mixture-of-Experts? . . . . .                         | 2        |
| 1.4      | Our question and contribution . . . . .                     | 2        |
| <b>2</b> | <b>Background concepts (from scratch)</b>                   | <b>3</b> |
| 2.1      | Spiking neurons: LIF and QIF . . . . .                      | 3        |
| 2.2      | Training spikes: surrogate gradients . . . . .              | 3        |
| 2.3      | Mixture-of-Experts: soft (1991) vs. sparse (2017) . . . . . | 3        |
| <b>3</b> | <b>Method: the surrogate in full detail</b>                 | <b>3</b> |
| 3.1      | Rollout backbone . . . . .                                  | 3        |
| 3.2      | QIF spiking neuron (exact discrete form) . . . . .          | 4        |
| 3.3      | Dense block (baseline) . . . . .                            | 4        |
| 3.4      | Soft-gated MoE block (our method) . . . . .                 | 4        |
| 3.5      | Parameter accounting (derivation) . . . . .                 | 5        |
| <b>4</b> | <b>Benchmarks: the PDEs and how data is made</b>            | <b>5</b> |
| <b>5</b> | <b>Experimental protocol</b>                                | <b>6</b> |
| <b>6</b> | <b>Metrics (defined and motivated)</b>                      | <b>6</b> |
| <b>7</b> | <b>Results</b>                                              | <b>6</b> |
| <b>8</b> | <b>Discussion, scope, and honest limitations</b>            | <b>7</b> |

---

## 1 Introduction and problem statement

### 1.1 What is a PDE surrogate?

Many physical systems (fluids, heat, traffic, phase separation) are described by *partial differential equations* (PDEs): equations relating a field  $u(x, t)$  (e.g. velocity, temperature, density) to its

derivatives in space  $x$  and time  $t$ . Classical numerical solvers integrate a PDE by taking many tiny time steps on a fine grid, which is accurate but expensive. A *neural surrogate* is a neural network trained to imitate the solver: given the field now, predict the field a moment later, and iterate. Because it can take larger steps and run on a GPU, it is potentially much faster.

We consider *autoregressive rollout* surrogates. Let  $u^t \in \mathbb{R}^{N_c \times N_x}$  denote the field at discrete time  $t$ , sampled on  $N_x$  spatial points with  $N_c$  physical channels (e.g.  $N_c=1$  for a scalar field,  $N_c=2$  for a two-component system). A rollout surrogate learns a map

$$u^{t+1} = \mathcal{F}_\theta(u^t), \quad (1)$$

and produces a trajectory by feeding its own output back in:  $u^t \rightarrow u^{t+1} \rightarrow u^{t+2} \dots$ . This “closed loop” is powerful but fragile: small per-step errors accumulate and can blow up over a long rollout, so stability is a central concern.

## 1.2 Why spiking neurons?

Standard neural networks (“artificial neural networks”, ANNs) use continuous activations (ReLU, GELU). *Spiking neural networks* (SNNs) instead use neurons that emit discrete 0/1 “spikes” when an internal state crosses a threshold, mimicking biological neurons. Their appeal is *energy*: on neuromorphic hardware, a neuron that does not spike costs (almost) nothing, so sparse spiking can be very cheap. The trade-off is that spikes are discrete and harder to train, and SNNs are usually *less accurate* than ANNs of the same size. Our codebase uses a specific spiking neuron, the *quadratic integrate-and-fire* (QIF) neuron (Section 3.2), for all nonlinearities.

## 1.3 Why a Mixture-of-Experts?

A *Mixture-of-Experts* (MoE) replaces one network (“dense”) by several smaller sub-networks (“experts”) plus a small *router* that decides, per input, how much each expert contributes. The intuition is *specialization*: different experts can handle different kinds of input. For a PDE with a moving *shock* (a sharp, nearly discontinuous front) sitting in an otherwise *smooth* field, one might hope that one expert learns to handle the shock and another the smooth background. This report tests exactly that hope.

## 1.4 Our question and contribution

*Does a spiking MoE, by specializing its experts to shock vs. smooth regions, outperform a spiking dense surrogate of comparable size on autoregressive PDE rollout?* Our findings:

1. On advective **shock**-forming conservation laws (Burgers, Buckley–Leverett) the MoE beats a parameter-matched spiking dense baseline using *fewer* parameters ( $\sim 24\%$ ), generalizing to held-out (unseen) PDE parameters.
2. The advantage is caused by *input-dependent routing*, not by merely having more branches: replacing the learned router by uniform weights erases the gain on shock PDEs but not elsewhere.
3. The experts specialize *spatially*: one tracks the moving shock, the other the smooth region.
4. The advantage is *narrow and predictable*: it appears only when there is a moving shock/smooth contrast *and* the dense baseline is capacity-limited. On smooth, elliptic, forced, or “easy” PDEs the MoE ties the dense baseline (it never loses to it in our final configuration).

## 2 Background concepts (from scratch)

### 2.1 Spiking neurons: LIF and QIF

A spiking neuron maintains a scalar *membrane potential*  $v$  that integrates its input; when  $v$  reaches a threshold  $\theta$  it emits a spike and resets. The simplest model is the *leaky integrate-and-fire* (LIF):  $\tau \dot{v} = -v + I$ , i.e.  $v$  relaxes linearly toward the input  $I$ . The *quadratic integrate-and-fire* (QIF) neuron adds a quadratic term:

$$\tau \dot{v} = a v^2 + \eta + I. \quad (2)$$

This quadratic form is the *normal form of a saddle-node bifurcation* and describes “Type-I” excitability: for  $\eta < 0$  there are two fixed points (a stable rest state and an unstable threshold), and as the input  $I$  increases they collide and disappear, after which  $v$  accelerates to  $+\infty$  in finite time — a spike. Intuitively, QIF has a genuine, sharp firing onset (unlike the soft LIF), which is why it is a richer, more nonlinear neuron. We use QIF because it is the neuron in the codebase; Section 3.2 gives our exact discrete-time version.

### 2.2 Training spikes: surrogate gradients

The spike  $s = H(v - \theta)$  (Heaviside step) has zero derivative almost everywhere, so ordinary back-propagation cannot train through it. The standard fix is a *surrogate gradient*: use the true 0/1 spike in the forward pass, but pretend, in the backward pass, that  $s$  is a smooth function of  $v$  with a bump-shaped derivative peaked at  $v = \theta$ . This lets gradients flow while keeping spikes discrete at inference.

### 2.3 Mixture-of-Experts: soft (1991) vs. sparse (2017)

The *original* MoE (Jacobs, Jordan, Nowlan & Hinton, 1991) is *soft*: a gating network outputs a softmax weight for *every* expert, and the output is the weighted sum of *all* experts. Every expert runs on every input. The *modern* MoE (Shazeer et al., 2017; Switch Transformer; Mixtral) is *sparse*: the router picks only the top- $k$  experts ( $k \ll E$ ), so most experts are skipped and compute stays constant as  $E$  grows — this is the “efficient MoE” people usually mean. **Our method is the soft, 1991-style MoE** (all experts active, input-dependent weights). We found that sparse top- $k$  routing did *not* help in our setting; only soft gating did. Its efficiency therefore comes from being a *parameter-efficient* multi-expert factorization, not from conditional-compute sparsity — we are explicit about this to avoid over-claiming.

## 3 Method: the surrogate in full detail

### 3.1 Rollout backbone

The field enters and leaves through 1D convolutions (kernel size 5, “same” padding), with a *residual* (skip) connection so the network predicts the *change* rather than the full next state (this is standard for rollout and eases stability). With  $C=48$  hidden channels,

$$h = W_1 * u^t \in \mathbb{R}^{C \times N_x}, \quad u^{t+1} = \mathcal{F}(u^t) = u^t + W_2 * \Phi(h), \quad (3)$$

where  $W_1 : N_c \rightarrow C$ ,  $W_2 : C \rightarrow N_c$  are convolutions and  $\Phi$  is the block (dense or MoE). The convolution gives each output location a small spatial receptive field; the block  $\Phi$  is where the spiking nonlinearity and (for the MoE) the routing live.

### 3.2 QIF spiking neuron (exact discrete form)

We use a one-step Euler discretization of Eq. (2) with a per-timestep reset. For drive  $x$  (a channel of  $h$ ) and membrane state  $v$  (carried across rollout steps, initialized to 0):

$$\Delta v = (a v^2 + \eta + x) \frac{\Delta t}{\tau}, \quad (4)$$

$$v \leftarrow v + \Delta v, \quad (5)$$

$$s = H(v - \theta) \quad (\text{spike, 0/1}), \quad (6)$$

$$v \leftarrow v(1 - s) + v_{\text{reset}} s. \quad (7)$$

Equation (4)–(5) integrates the input with the quadratic self-drive  $av^2$ ; (6) fires when the threshold is crossed; (7) resets fired units to  $v_{\text{reset}}$  and leaves the rest unchanged. The learnable parameters (with defaults) are

$$a = 1, \quad \eta = 0, \quad \tau = 2, \quad \Delta t = 1, \quad \theta = 1, \quad v_{\text{reset}} = -1,$$

all trained by gradient descent. The block output is the spike tensor  $s$ .

**Surrogate gradient.** Forward uses the exact step  $s = H(v - \theta)$ . Backward uses

$$\frac{\partial s}{\partial v} \approx p \exp(-w |v - \theta|), \quad (8)$$

a bump peaked at threshold, with learnable width  $w$  and peak  $p$  (initialized  $w=2, p=1$ ).

### 3.3 Dense block (baseline)

A single QIF feed-forward “expert” with expansion ratio  $R$ : project up to width  $RC$ , apply the QIF neuron, project back to  $C$ :

$$\Phi_{\text{dense}}(h) = W_{\downarrow} \text{QIF}(W_{\uparrow} h), \quad W_{\uparrow}: C \rightarrow RC, \quad W_{\downarrow}: RC \rightarrow C. \quad (9)$$

These  $W_{\uparrow}, W_{\downarrow}$  are  $1 \times 1$  convolutions (i.e. per-location linear maps). Our main baseline is **dense-R8** ( $R=8$ , hidden width  $8C=384$ ).

### 3.4 Soft-gated MoE block (our method)

We use  $E$  experts, each an up-projection  $W_{\uparrow}^{(e)}: C \rightarrow RC$ , plus a router  $W_r: C \rightarrow E$ , one *shared* QIF neuron, and one down-projection  $W_{\downarrow}: RC \rightarrow C$ . At every spatial location  $x$ :

$$g(x) = \text{softmax}(W_r h(x)) \in \mathbb{R}^E \quad (\text{per-location gate; } \sum_e g_e = 1), \quad (10)$$

$$m(x) = \sum_{e=1}^E g_e(x) (W_{\uparrow}^{(e)} h(x)) \in \mathbb{R}^{RC} \quad (\text{soft, all-expert mixture}), \quad (11)$$

$$\Phi_{\text{MoE}}(h) = W_{\downarrow} \text{QIF}(m). \quad (12)$$

Key points a first-time reader should note:

- The gate  $g(x)$  depends on the *local field content*  $h(x)$ , so *different spatial locations weight the experts differently* — this is the “routing”.

- All experts contribute (soft / top- $E$ ); we are not skipping any. This is the 1991 MoE.
- The experts differ only in  $W_{\uparrow}^{(e)}$ ; they share the QIF and the down-projection, which keeps the parameter count small.

Our final configuration is **E2-soft-R4**:  $E=2$  experts, ratio  $R=4$ .

### 3.5 Parameter accounting (derivation)

With  $N_c=1$ ,  $C=48$ , kernel 5, count weights+biases:

$$\begin{aligned} W_1 &: 1 \cdot C \cdot 5 + C, & W_2 &: C \cdot 1 \cdot 5 + 1, \\ W_r &: C \cdot E + E, & W_{\downarrow} &: RC \cdot C + C, & W_{\uparrow}^{(e)} &: C \cdot RC + RC \quad (\times E). \end{aligned}$$

Summing, the MoE and dense totals are

$$P_{\text{MoE}}(E, R) = 2352 ER + 2304 R + 49 E + 583, \quad P_{\text{dense}}(R) = 4656 R + 583. \quad (13)$$

Hence **E2-soft-R4** = 28,713 parameters versus **dense-R8** = 37,831 — the MoE is 24% smaller. (For top- $k$  sparse variants one would replace  $E$  by  $k$  in the *active*, i.e. per-forward, count; we report soft gating where active = total.)

## 4 Benchmarks: the PDEs and how data is made

All datasets are generated by the lab solver suite, which follows the *CAACT* convention (the same benchmark family used by the spiking-SciML codebase). For each PDE we sweep one physical parameter over 12 trajectories to create *heterogeneity* (a range of regimes), and we evaluate on *held-out* parameter values (interleaved, never seen in training) to test generalization. All fields are resampled to  $N_x=128$  and normalized to zero mean, unit variance using training statistics.

**Burgers (advective shock).**  $\partial_t u + u \partial_x u = \nu \partial_{xx} u$ , viscosity  $\nu \in [2, 20] \times 10^{-3}$ . The nonlinear advection  $u \partial_x u$  steepens the profile into a *moving shock*; the viscosity  $\nu$  sets its sharpness. Sharp front on a smooth background — the canonical case for our mechanism.

**Buckley–Leverett (CAACT form).**  $\partial_t u = \partial_{xx}(2u^2 + \frac{4}{3}u^3)$  on  $[0, 1]$  with Dirichlet boundaries. A degenerate nonlinear diffusion that develops a sharp front; the heterogeneity parameter is the initial-condition width.

**Heat (smooth).**  $\partial_t u = \alpha \partial_{xx} u$ ,  $\alpha \in [5, 50] \times 10^{-3}$ , with the analytic  $\sin(\pi x)$  Dirichlet mode. Pure diffusion: the field only smooths out; there is *no* sharp feature. This is a negative control: the mechanism should give *no* advantage.

**Navier–Stokes 1D (forced Burgers).**  $\partial_t u + u \partial_x u = \nu \partial_{xx} u + f(x, t)$  with a random time-varying body force  $f$ . The forcing keeps the whole field active, *removing* the quiescent smooth region — a useful contrast to plain Burgers.

**LWR traffic (scalar conservation law).**  $\partial_t \rho + \partial_x(\rho(1 - \rho)) = 0$ . The Lighthill–Whitham–Richards model of traffic; a localized jam steepens into a moving shock. (Implemented here with a Rusanov finite-volume scheme; note this generator is *not* part of the CAACT suite.)

## 5 Experimental protocol

- **Data.** 12 trajectories per PDE (parameter sweep); train on the trajectories, evaluate free rollout on held-out parameter values. Training horizon  $T_{\text{tr}}=45$  steps; evaluation rollout  $T=55$  steps ( $> T_{\text{tr}}$ , testing extrapolation in time).
- **Loss (teacher forcing).** One-step mean-squared error on the training horizon:

$$\mathcal{L} = \frac{1}{T_{\text{tr}} - 1} \sum_{t=1}^{T_{\text{tr}}-1} \|\mathcal{F}(u^t) - u^{t+1}\|_2^2. \quad (14)$$

“Teacher forcing” means each training step is fed the *true*  $u^t$ , not the model’s own prediction.

- **Optimizer.** AdamW, learning rate  $10^{-3}$  with cosine decay to  $10^{-5}$ , weight decay  $10^{-5}$ , gradient-norm clipping at 1.0, 1500 epochs. (These follow standard neural-PDE practice; a fixed schedule is applied identically to *both* dense and MoE for fairness.)
- **Repeats and significance.** 5 random seeds; the main tables report the mean relative  $L_2$  and a Welch  $t$ -statistic for the dense–MoE difference ( $|t| > 2 \approx$  significant). **Seed 42 is the canonical reference seed:** released model weights and all figures are produced with seed 42 (its held-out errors, e.g. Burgers 0.097, Buckley 0.064, Heat 0.133, NS 0.186, LWR 0.163, are within seed variation of the 5-seed means reported below).

## 6 Metrics (defined and motivated)

**Relative  $L_2$  error.** Accuracy of the free rollout:  $\text{relL2} = \|\hat{u} - u\|_2 / \|u\|_2$ . A value  $\gtrsim 1$  means the prediction is no better than zero (diverged); good surrogates are well below 0.1.

**Routing-gain (causal control).** To prove that the benefit comes from *input-dependent routing* rather than from simply having several branches, we retrain the MoE with the router *disabled* — fixed uniform gates  $g_e \equiv 1/E$  — and measure

$$\text{routing-gain} = \text{relL2}_{\text{uniform}} - \text{relL2}_{\text{learned}}. \quad (15)$$

A large positive value means the learned router is doing real, causal work.

**Specialization correlation.** To see *where* experts act, we correlate each expert’s time-averaged spatial gate usage  $\bar{g}_e(x)$  with the local sharpness  $|\partial_x u|(x)$ :  $\text{corr}(\bar{g}_e(x), |\partial_x u|(x))$ . A strong positive/negative pair means one expert concentrates on the sharp front and the other on the smooth region.

## 7 Results

**Efficiency (Pareto view).** On Burgers, spiking dense saturates with width — relative  $L_2$  of 0.175, 0.144, 0.144 at  $R=2, 4, 8$  (params 9.9k, 19.2k, 37.8k) — while the MoE reaches 0.111 at 28.7k, i.e. *below* the dense curve at fewer parameters. Buckley behaves the same. So the MoE is not merely “a bigger network”; it sits above the dense capacity frontier.

Table 1: Main comparison: our E2-soft-R4 MoE (28.7k params) vs. spiking dense-R8 (37.8k params). Held-out evaluation, 5 seeds; lower relative  $L_2$  is better.  $t$  is Welch’s statistic for (dense – MoE).

| Benchmark        | character                   | MoE          | dense-R8 | $t$   | verdict    |
|------------------|-----------------------------|--------------|----------|-------|------------|
| Burgers          | advective shock             | <b>0.111</b> | 0.144    | 3.54  | <b>WIN</b> |
| Buckley–Leverett | advective shock             | <b>0.075</b> | 0.094    | 3.33  | <b>WIN</b> |
| Heat             | smooth (control)            | 0.133        | 0.133    | −0.34 | tie        |
| Navier–Stokes    | forced (no quiescence)      | 0.195        | 0.210    | 0.72  | tie        |
| LWR              | scalar shock (unsat. dense) | 0.195        | 0.197    | 0.15  | tie        |

Table 2: Causal control and mechanism. Routing-gain isolates the learned router; the specialization correlation shows experts split shock vs. smooth.

| Benchmark        | routing-gain | $t$  | $ \text{corr}(\bar{g},  \partial_x u ) $ |
|------------------|--------------|------|------------------------------------------|
| Burgers          | +0.037       | 4.48 | 0.87                                     |
| Buckley–Leverett | +0.024       | 4.08 | 0.42                                     |
| LWR              | +0.041       | 3.56 | 0.95                                     |
| Heat             | 0.000        | 0.02 | (n/a; smooth)                            |

### Interpretation.

1. **The MoE wins only on advective shock PDEs**, and does so with fewer parameters than the dense baseline. On smooth/forced PDEs it ties.
2. **Routing is causal.** Disabling the router (uniform gates) erases the gain on shock PDEs (routing-gain  $t > 3.5$ ) but changes nothing on Heat (routing-gain  $\approx 0$ ). So the advantage is the *input-dependent* gating, not the extra branches.
3. **Experts specialize spatially** to shock vs. smooth (strong  $\pm$  correlations).
4. **Specialization is necessary but not sufficient.** On LWR the router clearly helps the MoE internally (routing-gain +0.041) and specialization is strongest (0.95), yet the MoE only *ties* the dense baseline — because on LWR the dense baseline is not capacity-saturated (adding width keeps helping it). The win over dense therefore needs *both* a routable shock/smooth contrast *and* a capacity-limited dense baseline.

## 8 Discussion, scope, and honest limitations

- **Where it works.** Moving shock against a quiescent smooth background, with a capacity-limited dense baseline (Burgers, Buckley). This is a *precise, falsifiable* applicability boundary, which we regard as the main contribution rather than a single number.
- **Where it does not.** Smooth diffusion (Heat), elliptic problems, forced flows without a quiescent region (NS), and shock problems that are either easy or where the dense baseline is unsaturated (LWR) — all ties.
- **Soft, not sparse.** Our gains come from the classical soft-gated (1991) MoE; sparse top- $k$  routing did not help. We therefore claim *parameter* efficiency (fewer stored / per-step parameters at equal or better accuracy), *not* the conditional-compute sparsity of modern sparse MoE.

- **Provenance.** Burgers, Buckley, Heat, and NS follow the CAACT lab-solver convention; the LWR generator is our own finite-volume implementation and is flagged as such.